Parcial 3, Parte 2: Physical Design

Arquitectura de Computadoras, Universidad del Istmo de Guatemala

José Rueda, carné 18123

Aviso importante

El servidor de Synopsys (VCS, Verdi, DC shell e IC Compiler II) no estuvo disponible el día de la entrega, así que no pude hacer la Parte 2 con esas herramientas. Hice el mismo flujo con Vivado 2025.1 como equivalente. No es lo mismo: Vivado es un flujo de FPGA y Synopsys es de ASIC, y eso cambia varias cosas, que explico en cada inciso. Dejo esta tabla de qué usé en lugar de qué:

Lo que pide el parcial	**Lo que usé en Vivado**
VCS + Verdi	xsim (simulador de Vivado) con testbench
DC shell + SDC	Síntesis de Vivado + archivo XDC
ICC2 (floorplan, placement, CTS, routing)	Implementación de Vivado (opt, place, route)
GDSII	No existe en FPGA. Entrego el checkpoint (.dcp) y el netlist post-routing

1. Preparación

Armé alu_sync.sv, que es la ALU+ de 16 bits de la Parte 1 con registros de entrada (a, b y alu_control) y registros de salida (result y las banderas N, Z, C, V). El reset es asíncrono y activo en alto. Por los dos grupos de registros, el resultado sale 2 ciclos de reloj después de cambiar las entradas.

Lo verifiqué con tb_alu_sync.sv: reloj de 10 ns, 5,000 vectores aleatorios (con un 25 % de casos donde a = b para probar la bandera Z) y un modelo de referencia escrito aparte, con el retraso de 2 ciclos. Resultado: Errores: 0 (W=16).

Con VCS y Verdi no pude correrlo. El testbench ya trae el bloque para generar el dump de Verdi (con `ifdef FSDB), pero no lo ejecuté.

2. Síntesis

Escribí alu_sync.xdc con un reloj de 10 ns en el puerto clk, retardos de entrada de 2 ns (máximo) y 0.5 ns (mínimo) y retardos de salida de 2 ns. Es lo que hace el SDC en Synopsys. Mantuve la jerarquía (flatten_hierarchy none) para ver el área de cada bloque. Los reportes se generan con parte2_sintesis.tcl.

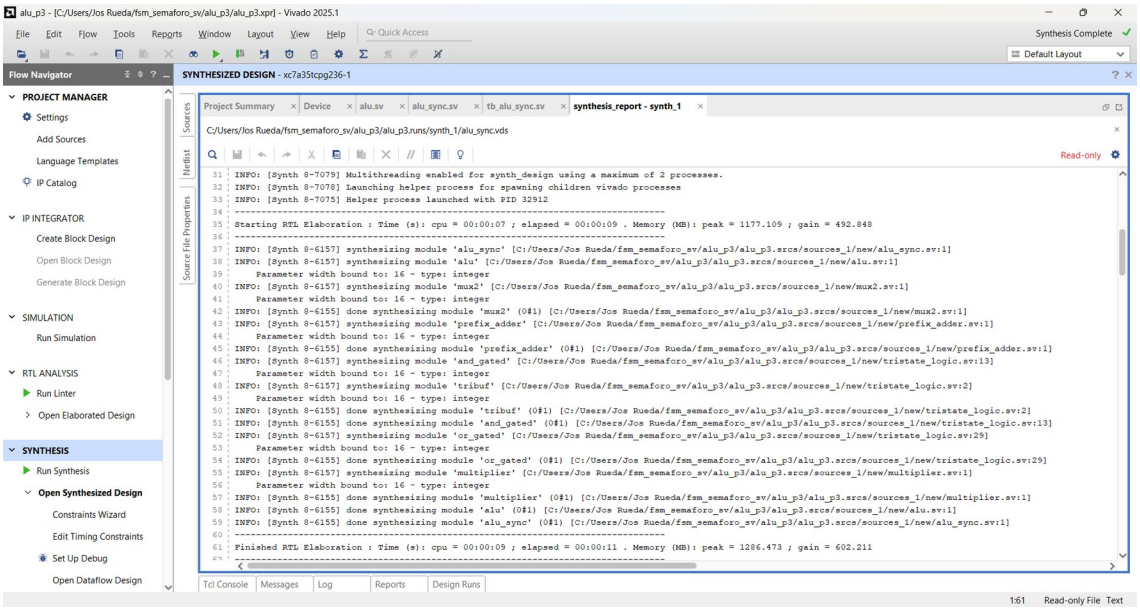


Figura 1. Vivado sintetizó todos los módulos con width = 16.

Resultados con el reloj de 10 ns:

- 499 LUTs (2.40 % del chip), 54 flip-flops (FDCE), 37 IBUF, 20 OBUF y 1 BUFG.
- WNS = −12.371 ns. El diseño no cumplía el timing.
- Ruta crítica: de a_r_reg[2]/C a z_reg/D. Sale del registro de entrada, atraviesa la ALU (el multiplicador con sus 15 sumadores encadenados y el mux de resultado) y llega al registro de la bandera Z, que además detecta si los 16 bits son cero. La ruta necesitaba unos 22 ns.

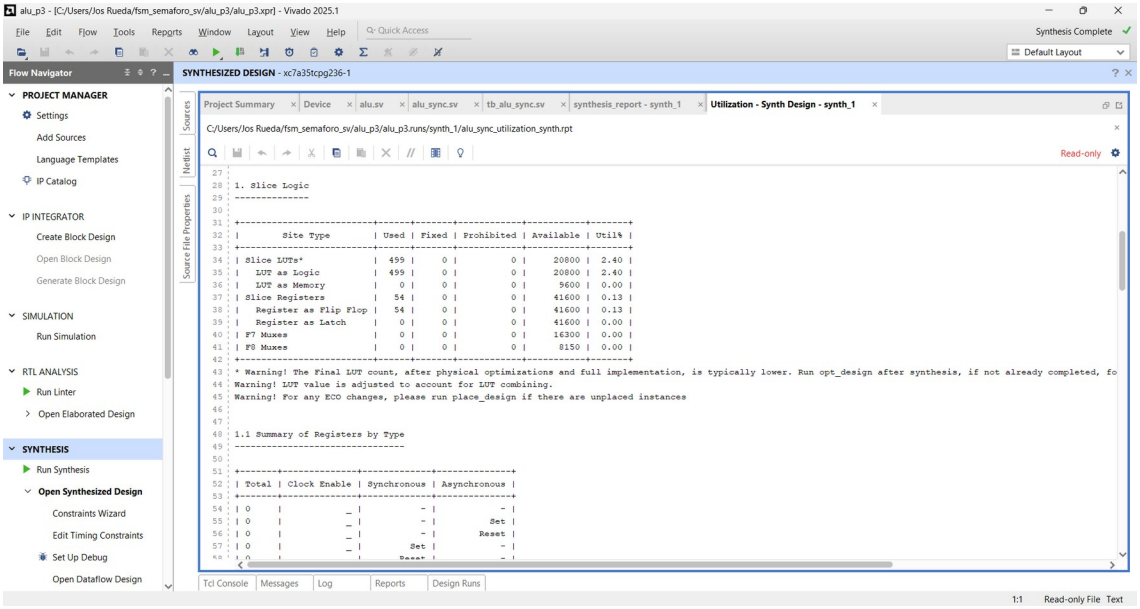


Figura 2. Reporte de utilización de la síntesis a 10 ns.

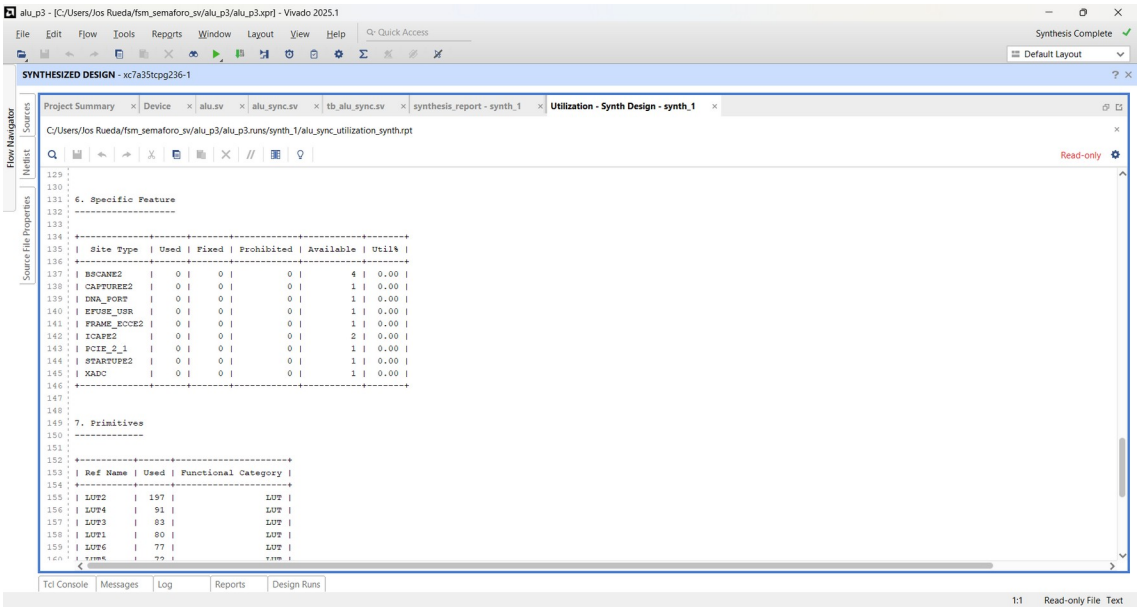


Figura 3. Primitivas usadas: solo LUTs, FDCE, IBUF, OBUF y BUFG.

Para corregir la violación subí el periodo a 40 ns (25 MHz) en el XDC y volví a sintetizar. Con eso el WNS pasó a +17.629 ns y las LUTs bajaron a 446, porque con un reloj más relajado la síntesis ya no necesita duplicar lógica para ir más rápido. Guardé los reportes de la primera corrida en la carpeta reportes_parte2_10ns.

3. Diseño físico

Corrí la implementación completa de Vivado con parte2_implementacion.tcl (optimización, placement, optimización física y routing). Lo equivalente a cada paso de ICC2:

- Floorplan: lo hizo Vivado automáticamente. No definí regiones (Pblocks).
- Alimentación: Vivado no tiene planificación de anillos de alimentación como ICC2. Lo que pude hacer es el análisis de potencia: 0.116 W en la corrida final (el 95 % de la parte dinámica es de los buffers de entrada y salida; el nivel de confianza es bajo porque no usé actividad de simulación).
- Placement: la lógica quedó agrupada en una zona compacta del chip, con los buffers de E/S repartidos por los bordes (Figura 4).
- CTS: en una FPGA el reloj viaja por la red global del chip a través de un BUFG (1 BUFG en el diseño). No se construye un árbol de reloj como en ICC2.
- Routing: 0 rutas fallidas.

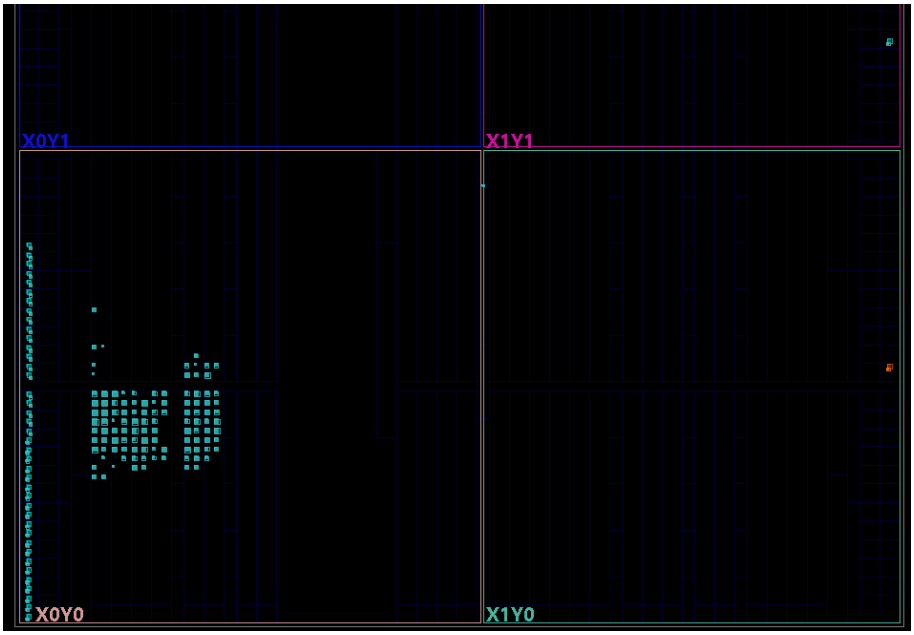


Figura 4. Device view con la ALU colocada (primera corrida de implementación).

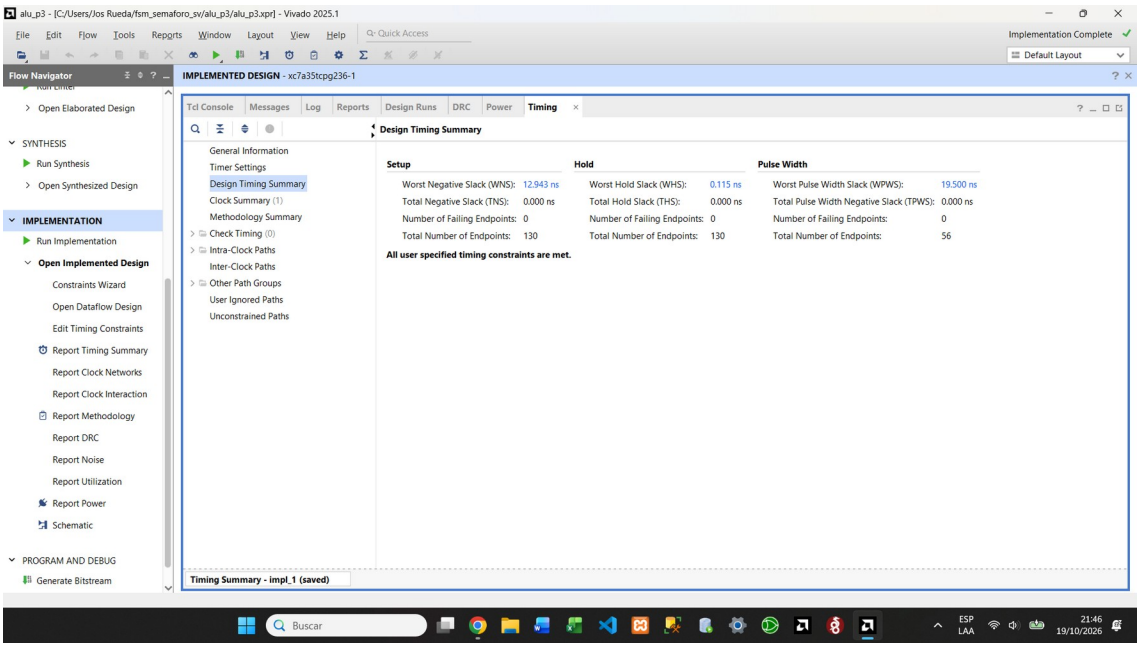


Figura 5. Resumen de timing de la primera corrida: WNS +12.943 ns, WHS +0.115 ns, sin endpoints en falla.

Violaciones físicas (DRC). La primera corrida dio 3 violaciones, todas de asignación de pines:

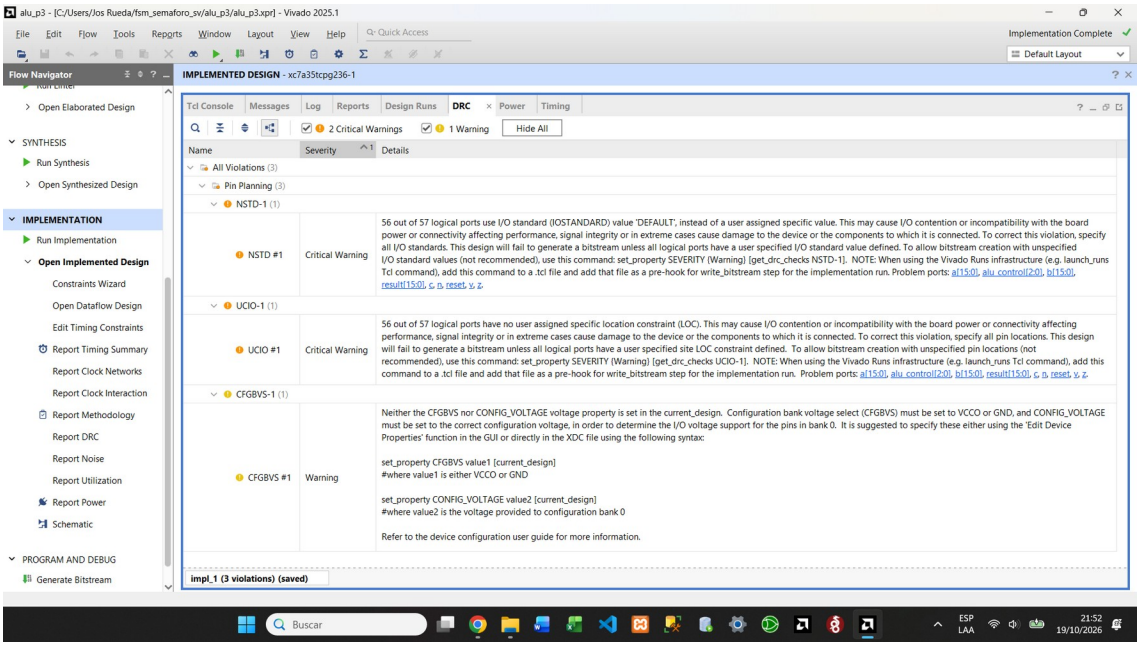


Figura 6. DRC antes de corregir: NSTD-1, UCIO-1 (críticos) y CFGBVS-1 (aviso).

Corregí NSTD-1 y CFGBVS-1 agregando al XDC el estándar LVCMOS33 para los puertos y los parámetros CFGBVS y CONFIG_VOLTAGE. Quedó solo UCIO-1 (puertos sin pin asignado). Esa no la pude corregir porque no estoy usando una placa específica, y solo impediría generar el bitstream, que no es parte del parcial.

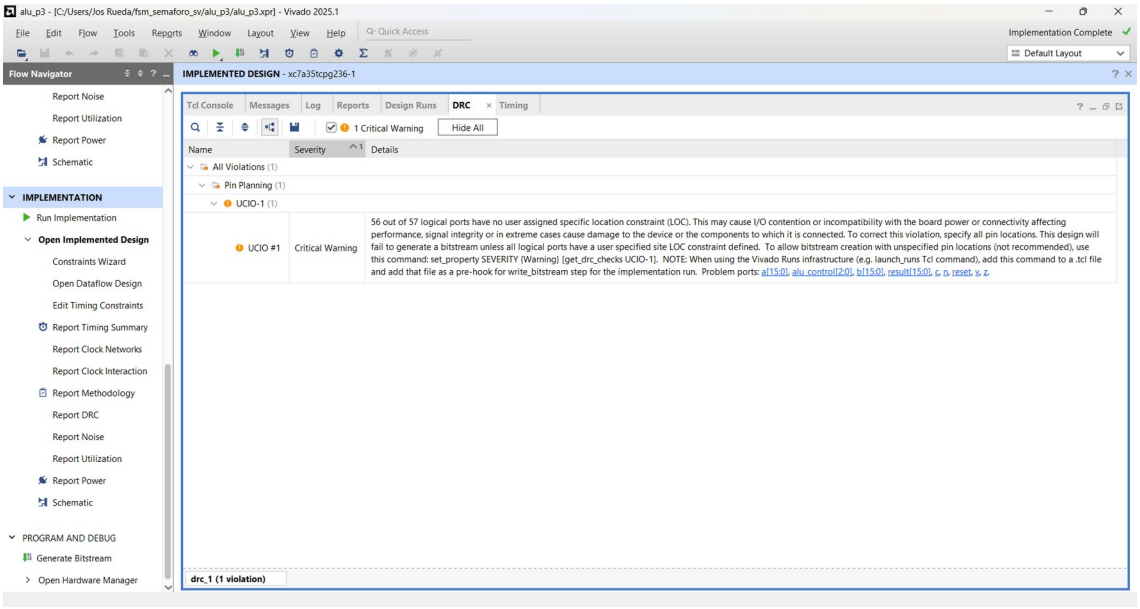


Figura 7. DRC después de corregir: solo queda UCIO-1.

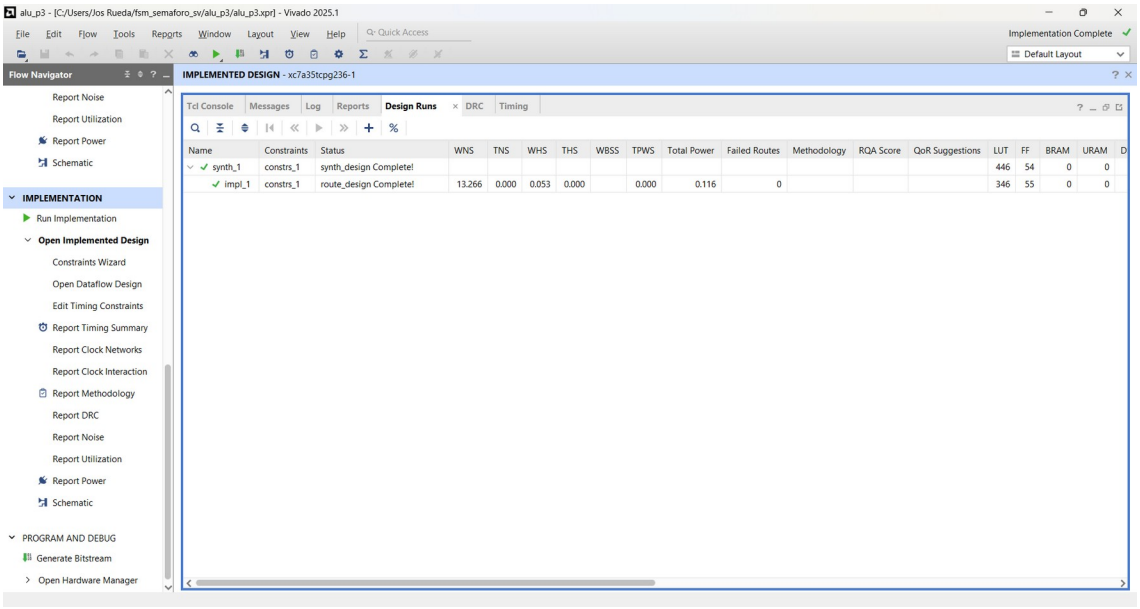


Figura 8. Resumen final de las corridas (synth_1 e impl_1).

4. Resultados

Métrica	Síntesis 10 ns	Síntesis 40 ns	Routing 40 ns (final)
WNS (setup)	−12.371 ns	+17.629 ns	+13.266 ns
WHS (hold)	n/a	n/a	+0.053 ns
LUTs	499	446	346
Flip-flops	54	54	55
Rutas fallidas	n/a	n/a	0
Potencia total	n/a	n/a	0.116 W

La primera corrida de routing (antes de corregir el DRC) dio WNS +12.943 ns, WHS +0.115 ns, 345 LUTs y 0.111 W. Las diferencias con la final salen de que al cambiar el estándar de E/S, Vivado volvió a hacer el placement y el routing.

El WNS baja de +17.629 a +13.266 ns porque en síntesis Vivado estima los retardos de las conexiones y después del routing usa los reales. Las LUTs bajan de 446 a 346 porque la implementación elimina lógica redundante y combina LUTs.

Del flip-flop de más (54 en síntesis y 55 en routing) no tengo una explicación confirmada.

GDSII: no existe en un flujo de FPGA. Lo equivalente que exporté es el checkpoint final (.dcp), el netlist post-routing y los reportes, que están en la carpeta reportes_parte2.

Cómo se transformaron los bloques del RTL:

- Los registros de alu_sync se convirtieron en celdas FDCE (flip-flop con reset asíncrono).
- Los mux2, los sumadores Prefix Adder, el multiplicador y las compuertas quedaron como LUTs. No aparecen CARRY4 ni DSP, porque describí los sumadores y el multiplicador con compuertas y no con los operadores + y *.
- Las entradas y salidas se volvieron IBUF y OBUF, y el reloj pasó por un IBUF y un BUFG.

- Buses tri-estado: en la lista de primitivas no aparece ninguna celda tri-estado (OBUFT, IOBUF o TBUF). Mi entendimiento es que Vivado eliminó los buffers tribuf, porque las FPGA no tienen tri-estado interno (solo en los pines), y dejó la lógica AND y OR. En un ASIC con Synopsys y una librería que tenga celdas tri-estado, esos buffers sí se podrían mapear.

5. Entregables

- RTL: carpeta rtl (mux2, prefix_adder, multiplier, tristate_logic, alu, alu_sync).
- Testbenches: carpeta tb.
- Scripts y restricciones: parte2_sintesis.tcl, parte2_implementacion.tcl y alu_sync.xdc.
- Reportes y netlists: reportes_parte2 (corrida final), reportes_parte2_10ns y reportes_parte2_antes_drc.
- Capturas: carpeta capturas_parte2.

6. Lo que no pude hacer

- Verificación con VCS y Verdi.
- Síntesis con DC shell (netlist, reportes y ruta crítica con librerías tecnológicas de Synopsys).
- Diseño físico con ICC2 y el GDSII.